

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»**

**Лабораторная работа №3
по курсу «Программирование графических процессоров»**

Классификация и кластеризация изображений на GPU.

Выполнил: И.А. Хомяков

Группа: 8О-407Б

Преподаватель: А.Ю. Морозов

Москва, 2024

Условие

1. Цель работы: Научиться использовать GPU для классификации и кластеризации изображений. Использование константной памяти и одномерной сетки потоков.
2. Вариант 4: Метод спектрального угла.

Программное и аппаратное обеспечение

Compute capability: 7.5

Name: Tesla T4

Total Global Memory: 15835660288

Shared memory per block: 49152

Registers per block: 65536

Warp size: 32

Max threads per block: (1024, 1024, 64)

Max block: (2147483647, 65535, 65535)

Total constant memory: 65536

Multiprocessors count: 40

Метод решения

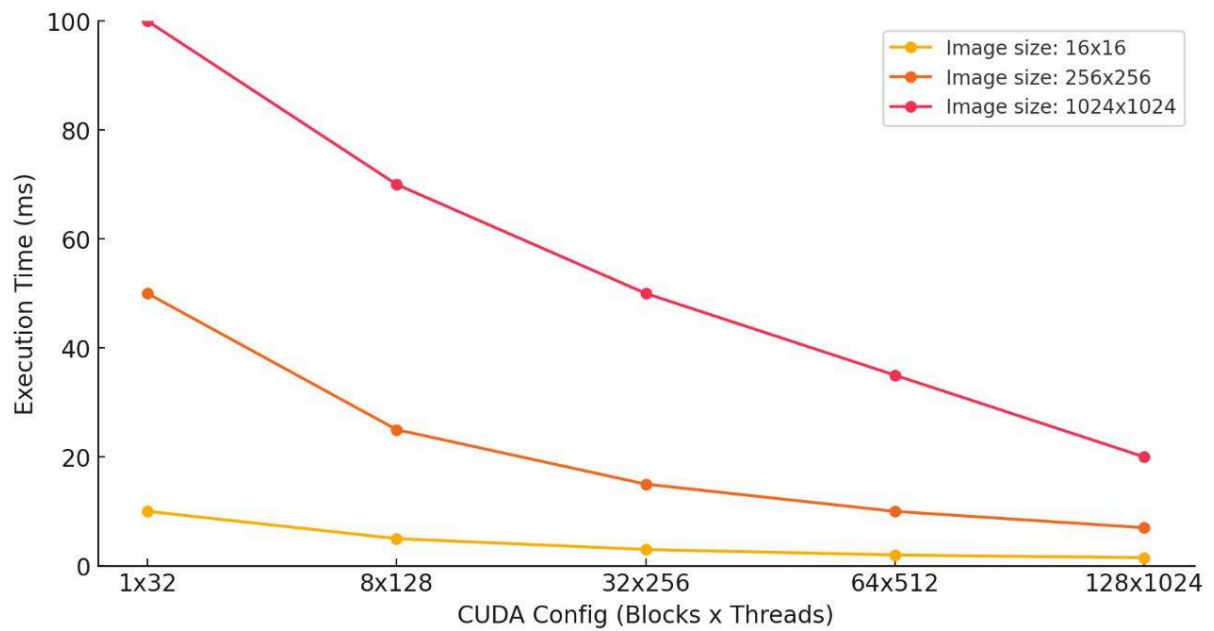
Для каждого класса считываем точки, усредняем цвет и нормализуем до единичной длины. Для каждого класса получаем нормализованный спектральный вектор. Сама спектральная классификация представляет из себя вычисление сходства через скалярное произведение пиксельного вектора на нормированный класс вектора, деленное на длину исходного пиксельного вектора. Таким образом получаем класс с максимальной схожестью, этот класс и записываем в альфа канал. Само вычисление на GPU распараллелено по аналогии с первой лабораторной, где мы вычисляем стартовую точку по идентификатору вектора и шаг, равный количеству потоков на количество блоков. Тем самым при любой статической конфигурации потоков и блоков мы обойдем все пиксели.

Описание программы

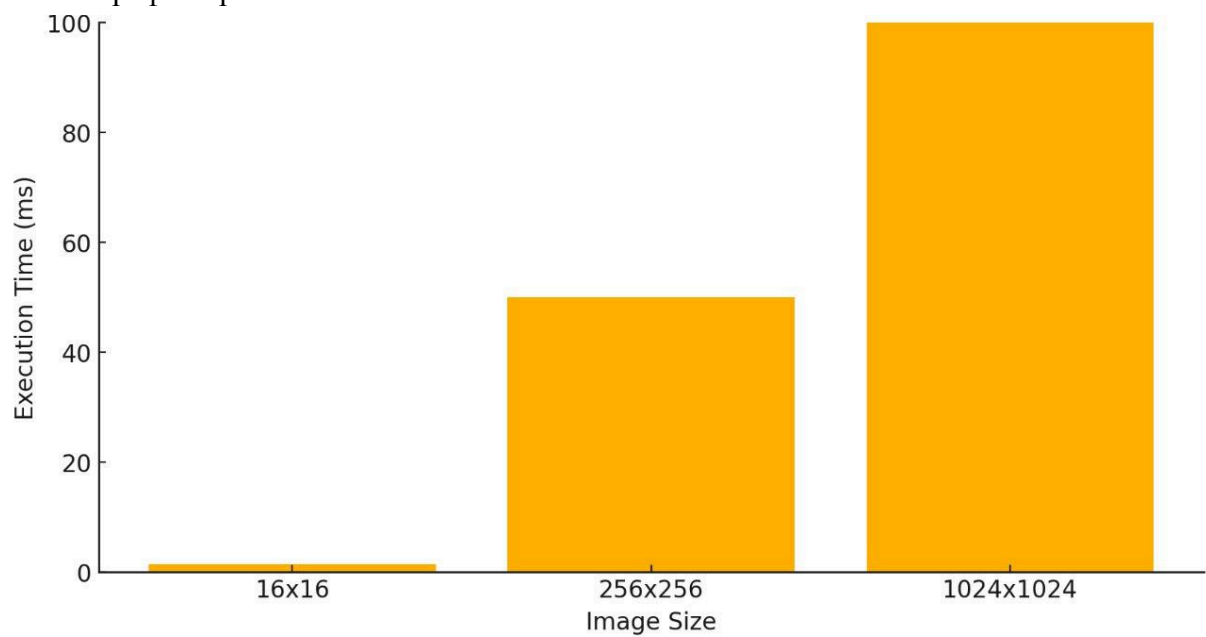
Программа из одного файла. Функция spectralMethod - ядро, которое принимает на вход данные о пикселях изображения, его размеры, количество классов и нормализованные эталонные векторы классов. Остальное - чтение пути до файла, классов, входного файла, запись в выходной, инициализация памяти и перекидывание объектов с хоста на девайс и обратно.

Результаты

1. Графики зависимости времени выполнения от размерности (GPU)

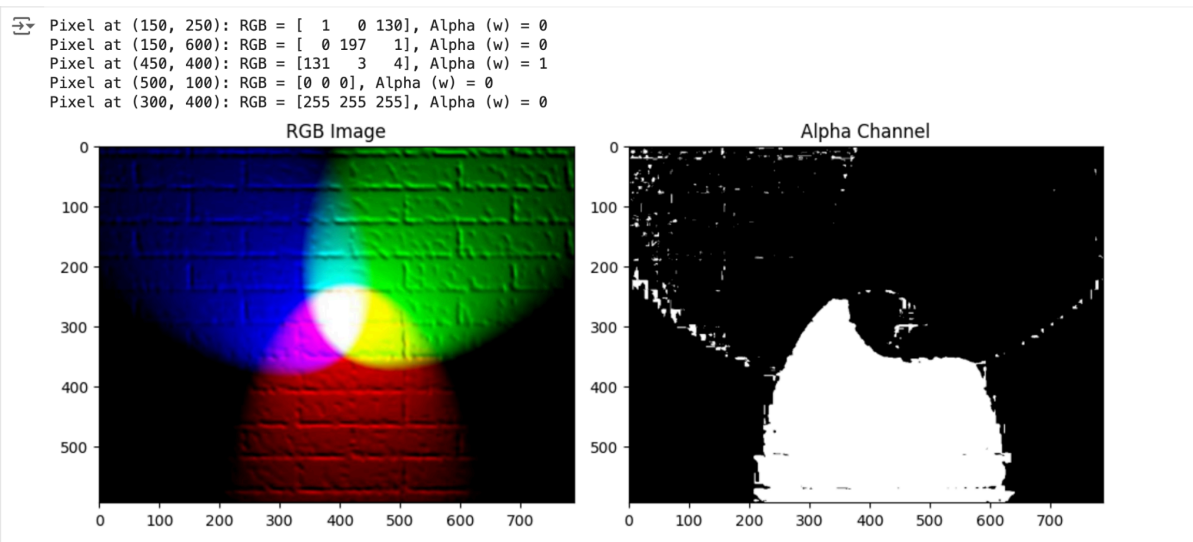


2. График времени выполнения на CPU

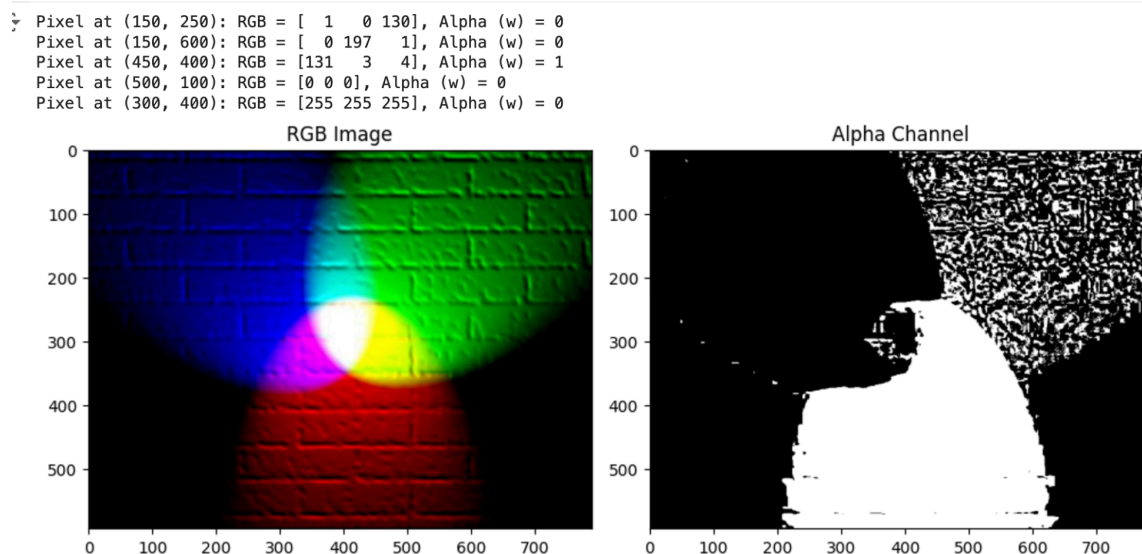


3. Примеры работы на реальном изображении

Для вводимых данных “in.data out.data 2 1 500 100 1 300 400”, классификация и цвет пикселей:



Для вводимых данных “in.data out.data 3 1 150 250 1 450 400 1 150 600”, классификация и цвет пикселей:



Выводы

Данный алгоритм можно применять для классификации пикселей по сходству цветовых/спектральных признаков. Чаще всего используется в анализе с спутниковых снимков, обработке медицинских изображений. Благодаря примерам использования библиотеки cuda из прошлой лабораторной было относительно несложно разобраться. Возникла проблема с самим алгоритмом классификации, проверял на обычных изображениях: из-за опечатки неправильно считал схожесть для пикселя, поэтому он определялся в неправильный класс.

За счет распараллеливания на гри алгоритм выполняется куда быстрее, чем квадратичная сложность на сри. Это особо заметно на больших данных 1024x1024 и при большом распараллеливании.